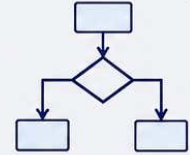
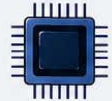


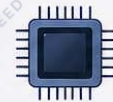
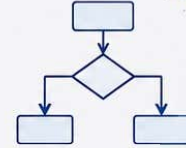
1010101010
0101010101
1010101010
0101010101



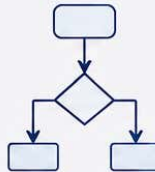
Notes



1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit-I

Binary Arithmetic

Binary arithmetic is the set of operations performed on binary numbers (0 and 1) using rules similar to decimal arithmetic.

1. Binary Number System

- Uses only two digits: 0 and 1.
- Base (radix) = 2
- Each position has a weight of 2^n .
- General form:

$$(b_{n-1} b_{n-2} \dots b_1 b_0)_2 = \sum_{i=0}^{n-1} b_i \times 2^i$$

Example:

$$(10110)_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ = 16 + 0 + 4 + 2 + 0 = (22)_{10}$$

2. Octal Number System

- Uses digits: 0 to 7
- Base = 8
- Each position has a weight 8^n .

Example:

$$(345)_8 = 3 \times 8^2 + 4 \times 8^1 + 5 \times 8^0 \\ = 192 + 32 + 5 = (229)_{10}$$

3. Hexadecimal Number System

- Uses digits: 0 to 9 and A to F
- Base = 16
- A=10, B=11, C=12, D=13, E=14, F=15
- Each position has weight 16^n .

Example:

$$(2AF)_{16} = 2 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 \\ = 512 + 160 + 15 = (687)_{10}$$

Common Hexadecimal Digits

Hex	Decimal	Hex	Decimal
0	0	8	8
1	1	9	9
2	2	A	10
3	3	B	11
4	4	C	12
5	5	D	13
6	6	E	14
7	7	F	15

4. Conversion between Number Systems

(a) Binary to Decimal

Example:

$$(1101)_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ = 8 + 4 + 0 + 1 = (13)_{10}$$

(b) Decimal to Binary

Example: Convert $(25)_{10}$ to binary

2	25	→ 1
2	12	→ 0
2	6	→ 0
2	3	→ 1
2	1	→ 1

$$(25)_{10} = (11001)_2$$

(c) Binary to Octal

- Group binary digits into 3 from right.

Example:

$$(110101)_2 = \underset{6}{110} \underset{5}{101} \\ = (65)_8$$

(d) Binary to Hexadecimal

- Group binary digits into 4 from right.

Example:

$$(11010110)_2 = \underset{D}{1101} \underset{6}{0110} \\ = (D6)_{16}$$

(e) Octal to Binary

- Replace each octal digit with 3-bit binary.

Example:

$$(67)_8 = \underset{110}{6} \underset{111}{7} \\ = (110111)_2$$

(f) Hexadecimal to Binary

- Replace each hex digit with 4-bit binary.

Example:

$$(3A)_{16} = \underset{0011}{3} \underset{1010}{A} \\ = (00111010)_2$$

5. Binary Addition

- Addition is performed using same rules as decimal, but in base 2.

$$\begin{aligned} 0 + 0 &= 0 \\ 0 + 1 &= 1 \\ 1 + 0 &= 1 \\ 1 + 1 &= 10 \text{ (write 0, carry 1)} \end{aligned}$$

Example:

$$\begin{array}{r} 1011 \\ + 0110 \\ \hline 10001 \end{array} \quad (1011)_2 + (0110)_2 = (10001)_2$$

6. Binary Subtraction

- Subtraction can be performed using 1's complement or 2's complement method.

Example: Subtract $(1011)_2$ from $(1101)_2$ using 2's complement.

1. 2's complement of $(1011)_2 \rightarrow (0101)_2$
2. Add to $(1101)_2$

$$\begin{array}{r} 1101 \\ + 0101 \\ \hline 10010 \end{array}$$

Result = $(0010)_2 = (2)_{10}$
(Discard carry)

7. 1's Complement

- 1's complement is obtained by changing 0 → 1 and 1 → 0.

Example:

$$(10110)_2 \rightarrow 1's \text{ complement} \rightarrow (01001)_2$$

8. 2's Complement

- 2's complement = 1's complement + 1

Example:

$$(10110)_2 \rightarrow 1's \text{ complement} = (01001)_2 \\ \text{Add } 1 \rightarrow (01010)_2 \\ \therefore 2's \text{ complement} = (01010)_2$$

9. Binary Multiplication

- Multiplication is done using repeated addition.

Example:

$$\begin{array}{r} 101 \\ \times 11 \\ \hline 101 \\ 101 \\ \hline 1111 \end{array} \quad \begin{aligned} &(101 \times 1) \\ &(101 \times 1, \text{ shifted left}) \end{aligned}$$

$$(101)_2 \times (11)_2 = (1111)_2$$

10. Binary Division

- Division is done using repeated subtraction or long division method.

Example: $(1101)_2 \div (10)_2$

$$\begin{array}{r} 110 \\ 10 \overline{) 1101} \\ \underline{110} \\ 101 \\ \underline{100} \\ 1 \end{array}$$

Quotient = $(110)_2$
Remainder = $(1)_2$

Binary, Octal Number System and their Inter Conversion

Different number systems, same value, different base.

1. Binary Number System

- Uses only two digits: 0 and 1.
- Base (radix) = 2
- Each position has a weight of 2^n .
- General form:

$$(b_{n-1} b_{n-2} \dots b_1 b_0)_2 = \sum_{i=0}^{n-1} b_i \cdot 2^i$$

Example:

$$(10110)_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ = 16 + 0 + 4 + 2 + 0 = (22)_{10}$$

2. Octal Number System

- Uses eight digits: 0 to 7.
- Base (radix) = 8
- Each position has a weight of 8^n .
- General form:

$$(o_{n-1} o_{n-2} \dots o_1 o_0)_8 = \sum_{i=0}^{n-1} o_i \cdot 8^i$$

Example:

$$(745)_8 = 7 \times 8^2 + 4 \times 8^1 + 5 \times 8^0 \\ = 448 + 32 + 5 = (485)_{10}$$

3. Place Value Comparison

Position (n)	Binary (2^n)	Octal (8^n)
0	1	1
1	2	8
2	4	64
3	8	512
4	16	4096

Note:

- Octal is a shorthand form of binary
- 1 Octal digit = 3 Binary digits (because $8 = 2^3$)

4. Octal and Binary Equivalents

Octal Digit	Binary (3 bits)
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

5. Binary to Octal Conversion

- Step 1:** Group the binary digits into 3 from right.
- Step 2:** If leftmost group has less than 3 digits, add leading 0's.
- Step 3:** Convert each 3-bit group into its equivalent octal digit.

Example: $(110101)_2 = (?)_8$

Group from right: $\begin{array}{ccc} 110 & 101 \\ \downarrow & \downarrow \\ 6 & 5 \end{array}$

$\therefore (110101)_2 = (65)_8$

If required, add leading zeros.

6. Octal to Binary Conversion

- Step 1:** Replace each octal digit with its 3-bit binary equivalent.
- Step 2:** Combine all 3-bit groups.

Example: $(374)_8 = (?)_2$

$$\begin{array}{ccc} 3 & 7 & 4 \\ \downarrow & \downarrow & \downarrow \\ 011 & 111 & 100 \end{array}$$

$\therefore (374)_8 = (011111100)_2$
 $= (11111100)_2$ (remove leading 0)

7. Binary to Decimal to Octal (Another Method)

Example: Convert $(101101)_2$ to octal.

Step 1: Convert binary to decimal

$$(101101)_2 = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ = 32 + 0 + 8 + 4 + 0 + 1 = (45)_{10}$$

Step 2: Convert decimal to octal

$$\begin{array}{r} 8 \overline{) 45} \\ \underline{5} \rightarrow R \ 5 \\ \underline{0} \rightarrow R \ 5 \end{array}$$

$\therefore (45)_{10} = (55)_8$
 $\therefore (101101)_2 = (55)_8$

This method is useful for checking the answer.

8. Octal to Decimal to Binary (Another Method)

Example: Convert $(257)_8$ to binary.

Step 1: Convert octal to decimal

$$(257)_8 = 2 \times 8^2 + 5 \times 8^1 + 7 \times 8^0 \\ = 128 + 40 + 7 = (175)_{10}$$

Step 2: Convert decimal to binary (repeated division)

$175 \div 2 \rightarrow R \ 1, 87 \div 2 \rightarrow R \ 1, 43 \div 2 \rightarrow R \ 1, 21 \div 2 \rightarrow R \ 1, \\ 10 \div 2 \rightarrow R \ 0, 5 \div 2 \rightarrow R \ 1, 2 \div 2 \rightarrow R \ 0, 1 \div 2 \rightarrow R \ 1$

$\therefore (175)_{10} = (10101111)_2$

$\therefore (257)_8 = (10101111)_2$

Division by 2, write remainders from bottom to top.

9. Direct Octal $\leftrightarrow$ Binary Conversion Examples

(i) $(726)_8$ to Binary

$$\begin{array}{ccc} 7 & 2 & 6 \\ \downarrow & \downarrow & \downarrow \\ 111 & 010 & 110 \end{array}$$

$\therefore (726)_8 = (111010110)_2$

(ii) $(100111)_2$ to Octal

Group: $\begin{array}{cc} 100 & 111 \\ \downarrow & \downarrow \\ 4 & 7 \end{array}$

$\therefore (100111)_2 = (47)_8$

10. Important Points

- Binary uses 2 digits: 0, 1
- Octal uses 8 digits: 0 to 7
- Octal is base 8, Binary is base 2
- 1 octal digit = 3 binary bits
- Group 3 bits for binary $\leftrightarrow$ octal conversion
- Add leading zeros if required
- Remove leading zeros in final answer
- Both direct conversion and decimal method can be used
- Always verify the result

Practice more problems for better understanding !

Hexadecimal Number System and their Inter Conversion

"Different number systems, same value, different base."

1. Hexadecimal Number System

- Uses 16 symbols : 0-9 and A-F.
- Base (radix) = 16
- Each position has a weight of 16^n .
- It is a compact way of representing binary numbers.
- Widely used in computer systems, memory addressing etc.

Symbols :

0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
A, B, C, D, E, F

Example :

$$(2AF)_{16} = 2 \times 16^2 + A \times 16^1 + F \times 16^0 \\ = 2 \times 256 + 10 \times 16 + 15 \times 1 \\ = 512 + 160 + 15 = (687)_{10}$$

2. Place Value Table of Hexadecimal

Position (n)	Power of 16 (16^n)	Value
0	16^0	1
1	16^1	16
2	16^2	256
3	16^3	4096
4	16^4	65536

Note :

- A = 10, B = 11, C = 12, D = 13, E = 14, F = 15
- Each leftward position is 16 times the previous position.

3. Hexadecimal to Decimal Conversion

Step 1 : Write each digit with its appropriate power of 16.

Step 2 : Replace A-F with their decimal values.

Step 3 : Calculate and add.

Example : $(3B7)_{16}$

$$= 3 \times 16^2 + B \times 16^1 + 7 \times 16^0 \\ = 3 \times 256 + 11 \times 16 + 7 \times 1 \\ = 768 + 176 + 7 \\ = (951)_{10}$$

4. Decimal to Hexadecimal Conversion

Step 1 : Divide the decimal number by 16.

Step 2 : Write the remainder in hexadecimal (0-9, A-F).

Step 3 : Repeat until the quotient becomes 0.

Step 4 : Write the remainders from bottom to top.

Example : $(785)_{10}$

16	785	R = 1 (1)
16	49	R = 1 (1)
16	3	R = 3 (3)
	0	

$$(785)_{10} \\ = (311)_{16}$$

5. Hexadecimal to Binary Conversion

- Replace each hexadecimal digit with its 4-bit binary equivalent.

Example : $(2AF)_{16}$

2	A	F
↓	↓	↓
0010	1010	1111

$$\therefore (2AF)_{16} = (001010101111)_2$$

Note :

- Each hexadecimal digit = 4 bits.
- So, number of binary bits = $4 \times$ (number of hexadecimal digits).

6. Binary to Hexadecimal Conversion

Step 1 : Group the binary digits into 4 bits from right (add leading zeros if needed).

Step 2 : Convert each 4-bit group to its hexadecimal digit.

Example : $(110101101111)_2$

1101	0110	1111
↓	↓	↓
D	6	F

$$\therefore (110101101111)_2 = (D6F)_{16}$$

Note :

- If the total number of bits is not a multiple of 4, add leading zeros.

7. Hexadecimal to Octal Conversion

Step 1 : Convert hexadecimal to binary.

Step 2 : Group the binary digits into 3 bits (from right).

Step 3 : Convert each group to octal digit.

Example : $(5C)_{16}$

$$5 \rightarrow 0101 \quad C \rightarrow 1100$$

Binary : 01011100

010	111	00
↓	↓	↓
5	7	0

$$\therefore (5C)_{16} = (134)_8$$

8. Octal to Hexadecimal Conversion

Step 1 : Convert octal to binary.

Step 2 : Group into 4 bits from right.

Step 3 : Convert each group to hexadecimal.

Example : $(725)_8$

$$7 \rightarrow 111 \quad 2 \rightarrow 010 \quad 5 \rightarrow 101$$

Binary : 111010101

0001	1101	0101
↓	↓	↓
1	D	5

$$\therefore (725)_8 = (1D5)_{16}$$

Key Points :

- Hexadecimal uses 16 symbols (0-9, A-F).
- 1 Hex digit = 4 binary bits.
- Useful in computer memory addressing.
- Conversion can be done via binary or directly using division method.
- Always verify the result.

Practice Examples :

1. Convert $(9E)_{16}$ to decimal.
2. Convert $(101101)_2$ to hexadecimal.
3. Convert $(247)_8$ to hexadecimal.
4. Convert $(3F)_{16}$ to binary and octal.

9. Shortcut Table for Conversions

Hex	Decimal	Binary (4 bits)	Binary (4 bits)	Hex	Decimal	Binary (4 bits)	Binary (4 bits)
0	0	0000	4 0100	8	8	1000	C 1100
1	1	0001	5 0101	9	9	1001	D 1101
2	2	0010	6 0110	A	10	1010	E 1110
3	3	0011	7 0111	B	11	1011	F 1111

"Same rules as decimal arithmetic, but in binary form."

Binary Arithmetic : Addition and Subtraction using 1's & 2's Complements

"Subtraction is performed by adding the complement of the subtrahend."

1. Binary Addition

- Follow the same rules as decimal addition.
- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 10$ (Carry 1)
- Add from right to left.
- If there is a carry out from the MSB, discard it (for fixed number of bits).

Example 1: Add two binary numbers.

$$\begin{array}{r} (1011)_2 + (1101)_2 \\ 1011 \\ + 1101 \\ \hline 11000 \end{array}$$

$$\therefore (1011)_2 + (1101)_2 = (11000)_2$$

Example 2: Add using 8-bit representation.

$$\begin{array}{r} (11001010)_2 + (00110101)_2 \\ 11001010 \\ + 00110101 \\ \hline 11111111 \end{array}$$

$$\therefore \text{Sum} = (11111111)_2$$

6. Comparison: 1's vs 2's Complement

Feature	1's Complement	2's Complement
How to get	Invert all bits	1's complement + 1
Representation of zero	Two zeros (+0 and -0)	Single zero
Used for	Subtraction	Subtraction, widely used in computers
End-around carry	Yes (in addition)	No
Hardware implementation	More complex	Simpler
Preferred in computers	Less	More

2. 1's Complement

- 1's complement is obtained by changing $0 \rightarrow 1$ and $1 \rightarrow 0$.
- Also called the "bitwise complement".
- Used for subtraction method.

Example: Find 1's complement.

- (i) $(1011001)_2$
1's complement $\rightarrow 0100110$
- (ii) $(0001110)_2$
1's complement $\rightarrow 1110001$

3. 2's Complement

- 2's complement is obtained by adding 1 to the 1's complement.
- Used widely in computers for subtraction because it gives a unique representation of 0.

Example: Find 2's complement.

- (i) $(1011001)_2$
1's complement $\rightarrow 0100110$
Add 1 $\rightarrow 0100111$
2's complement $\rightarrow 0100111$
- (ii) $(0001110)_2$
1's complement $\rightarrow 1110001$
Add 1 $\rightarrow 1110010$
2's complement $\rightarrow 1110010$

7. Key Points

- 1's complement: change $0 \rightarrow 1$ and $1 \rightarrow 0$.
- 2's complement: 1's complement + 1.
- Subtraction can be performed by adding the complement of the subtrahend.

4. Subtraction using 1's Complement Method

- Step 1:** Find 1's complement of the subtrahend.
- Step 2:** Add it to the minuend.
- Step 3:** If there is a carry out, add it to the LSB (end-around carry).
- Step 4:** If there is no carry, the result is negative. Take 1's complement of the result and add a negative sign.

Example 1: Subtract $(1101)_2 - (0110)_2$ using 1's complement.

$$\begin{array}{r} \text{Step 1: 1's complement of } 0110 \rightarrow 1001 \\ \text{Step 2: Add to } 1101 \\ 1101 \\ + 1001 \\ \hline 10010 \end{array}$$

← Carry 1

Step 3: Add carry to LSB $\rightarrow 0101 + 1 = 0110$

$$\therefore (1101)_2 - (0110)_2 = (0110)_2$$

Example 2: Subtract $(0100)_2 - (1101)_2$ using 1's complement.

$$\begin{array}{r} \text{Step 1: 1's complement of } 1101 \rightarrow 0010 \\ \text{Step 2: Add to } 0100 \\ 0100 \\ + 0010 \\ \hline 0110 \end{array}$$

(No carry)

Step 3: No carry $\rightarrow$ result is negative. Take 1's complement of $0110 \rightarrow 1001$.
 $\therefore$ Answer $= -(1001)_2$

5. Subtraction using 2's Complement Method

- Step 1:** Find 2's complement of the subtrahend.
- Step 2:** Add it to the minuend.
- Step 3:** If there is a carry out, discard it (result is positive).
- Step 4:** If there is no carry, result is negative. Take 2's complement of the result and add a negative sign.

Example 1: Subtract $(10010)_2 - (01101)_2$ using 2's complement.

$$\begin{array}{r} \text{Step 1: 2's complement of } 01101 \\ 1's \text{ complement} \rightarrow 10010 \\ \text{Add 1} \rightarrow 10011 \\ 2's \text{ complement} \rightarrow 10011 \end{array}$$

Step 2: Add to 10010

$$\begin{array}{r} 10010 \\ + 10011 \\ \hline 100101 \end{array}$$

(Carry 1)

Step 3: Discard carry $\rightarrow 00101$

$$\therefore (10010)_2 - (01101)_2 = (00101)_2$$

Example 2: Subtract $(00101)_2 - (01011)_2$ using 2's complement.

$$\begin{array}{r} \text{Step 1: 2's complement of } 01011 \\ 1's \text{ complement} \rightarrow 10100 \\ \text{Add 1} \rightarrow 10101 \\ 2's \text{ complement} \rightarrow 10101 \end{array}$$

Step 2: Add to 00101

$$\begin{array}{r} 00101 \\ + 10101 \\ \hline 11010 \end{array}$$

(No carry)

Step 3: No carry $\rightarrow$ negative result. Take 2's complement of $11010 \rightarrow 01101$.
 $\therefore$ Answer $= -(01101)_2$

- In 1's complement, use end-around carry.
- In 2's complement, discard the final carry.
- If no carry, result is negative $\rightarrow$ take complement and add (-) sign.
- 2's complement is more commonly used in computer systems.

"In binary arithmetic, subtraction is nothing but addition of a complement."

Multiplication is repeated addition.

Binary Arithmetic : Multiplication & Division

Division is repeated subtraction.

1. Binary Multiplication

- Multiplication of two binary numbers is performed using the same method as decimal multiplication.
- Multiply the multiplicand with each bit of the multiplier (from right to left).
- If the bit is 0 → write 0's.
- If the bit is 1 → write the multiplicand.
- Shift the partial product to the left for each next bit.
- Add all the partial products.

Example 1 : Multiply $(1011)_2$ by $(110)_2$

Step 1 : Write the numbers.

Multiplicand = 1011
Multiplier = 110

Step 2 : Multiply each bit of multiplier (from right to left).

$$\begin{array}{r}
 1011 \quad (\text{Multiplicand}) \\
 \times 110 \quad (\text{Multiplier}) \\
 \hline
 0000 \quad (1011 \times 0) \\
 1011 \quad (1011 \times 1) \\
 101100 \quad (1011 \times 1, \text{shifted left}) \\
 \hline
 1000010 \quad (\text{Product})
 \end{array}$$

$$\therefore (1011)_2 \times (110)_2 = (1000010)_2$$

Verification (Decimal):

$$(1011)_2 = 11, (110)_2 = 6$$

$$11 \times 6 = 66 \rightarrow (66)_{10} = (1000010)_2 \checkmark$$

Example 2 : Multiply $(1110)_2$ by $(101)_2$

$$\begin{array}{r}
 1110 \\
 \times 101 \\
 \hline
 1110 \quad (\times 1) \\
 0000 \quad (\times 0, \text{shifted}) \\
 111000 \quad (\times 1, \text{shifted}) \\
 \hline
 1000110
 \end{array}$$

$$\therefore (1110)_2 \times (101)_2 = (1000110)_2$$

Example 3 : Multiply $(1001)_2$ by $(1001)_2$

$$\begin{array}{r}
 1001 \\
 \times 1001 \\
 \hline
 1001 \quad (\times 1) \\
 0000 \quad (\times 0) \\
 0000 \quad (\times 0) \\
 1001000 \quad (\times 1, \text{shifted}) \\
 \hline
 1001001
 \end{array}$$

$$\therefore (1001)_2 \times (1001)_2 = (1001001)_2$$

Key Points (Multiplication)

- Follows the same procedure as decimal multiplication.
- Each partial product is either the multiplicand or 0.
- Shift left for each next bit.
- Add all partial products to get the final result.
- The product of an n-bit and m-bit number gives (n + m)-bit result.

1. Binary Division

- Division is performed using repeated subtraction or the long division method.
- At each step, compare the divisor with the leftmost bits of the dividend.
- If the divisor is less than or equal to the partial dividend, subtract and write 1 in quotient.
- If not, write 0 and bring down the next bit.
- Continue until all bits are used.

Example 1 : Divide $(110110)_2$ by $(101)_2$

$$\begin{array}{r}
 \text{Quotient} \rightarrow 110 \\
 101 \overline{) 110110} \\
 \underline{- 101} \\
 0011 \\
 \downarrow \text{bring next bit} \\
 0110 \\
 \underline{- 101} \\
 001 \rightarrow \text{Remainder}
 \end{array}$$

$$\therefore \text{Quotient} = (110)_2 \\ \text{Remainder} = (001)_2$$

Verification:

$$\begin{aligned}
 (101)_2 \times (110)_2 + (001)_2 \\
 = (110100)_2 + (001)_2 \\
 = (110110)_2 = (110110)_2 \checkmark
 \end{aligned}$$

Example 2 : Divide $(100101)_2$ by $(11)_2$

$$\begin{array}{r}
 \text{Quotient} \rightarrow 10011 \\
 11 \overline{) 100101} \\
 \underline{- 11} \\
 0100 \\
 \underline{- 00} \\
 100 \\
 \underline{- 11} \\
 11 \\
 \underline{- 11} \\
 00 \rightarrow \text{Remainder } 0
 \end{array}$$

$$\therefore \text{Quotient} = (10011)_2 \\ \text{Remainder} = (0)_2$$

Verification:

$$\begin{aligned}
 (11)_2 \times (10011)_2 + (0)_2 \\
 = (100101)_2 \checkmark
 \end{aligned}$$

$$\text{Dividend} = \text{Divisor} \times \text{Quotient} + \text{Remainder} \\ \text{Remainder} < \text{Divisor}$$

Example 3 : Divide $(1111)_2$ by $(10)_2$

$$\begin{array}{r}
 \text{Quotient} \rightarrow 111 \\
 10 \overline{) 1111} \\
 \underline{- 10} \\
 011 \\
 \underline{- 10} \\
 01 \rightarrow \text{Remainder}
 \end{array}$$

$$\therefore \text{Quotient} = (111)_2 \\ \text{Remainder} = (1)_2$$

Key Points (Division)

- Use long division method.
- Subtract only when the divisor is less than or equal to the partial dividend.
- Bring down the next bit after each step.
- Continue until all bits are used.
- Dividend = Divisor × Quotient + Remainder.
- Remainder is always less than the divisor.
- Division is useful in computer arithmetic, representing addresses, and algorithms.

"BCD represents each decimal digit with its 4-bit binary equivalent."

Binary Codes : 8421 BCD (Binary Coded Decimal)

"In 8421 BCD, the weights of the bits are 8, 4, 2 and 1."

1. Introduction

- BCD (Binary Coded Decimal) is a binary code used to represent decimal numbers.
- In 8421 BCD, each decimal digit (0-9) is represented by a 4-bit binary code.
- The weights of the bits from left to right are 8, 4, 2, 1 (hence 8421).
- It is a weighted code.
- Each decimal digit is coded separately (unlike pure binary).
- Widely used in digital systems such as calculators, digital clocks, and display devices (7-segment displays).

2. 8421 BCD Code Table

Decimal Digit	8	4	2	1	BCD Code
0	0	0	0	0	0000
1	0	0	0	1	0001
2	0	0	1	0	0010
3	0	0	1	1	0011
4	0	1	0	0	0100
5	0	1	0	1	0101
6	0	1	1	0	0110
7	0	1	1	1	0111
8	1	0	0	0	1000
9	1	0	0	1	1001

Note :

- 1010 to 1111 are invalid in 8421 BCD.
- Only 0000 to 1001 are used (for 0 to 9).
- Each decimal digit is represented by 4 bits.

3. Decimal to 8421 BCD Conversion

- Replace each decimal digit with its 4-bit BCD code from the table.

Example 1: Convert $(59)_{10}$ to 8421 BCD.

$$5 \rightarrow 0101 \quad 9 \rightarrow 1001$$

$$\therefore (59)_{10} = (0101 \ 1001)_{BCD}$$

Example 2: Convert $(247)_{10}$ to 8421 BCD.

$$2 \rightarrow 0010 \quad 4 \rightarrow 0100 \quad 7 \rightarrow 0111$$

$$\therefore (247)_{10} = (0010 \ 0100 \ 0111)_{BCD}$$

Example 3: Convert $(100)_{10}$ to 8421 BCD.

$$1 \rightarrow 0001 \quad 0 \rightarrow 0000 \quad 0 \rightarrow 0000$$

$$\therefore (100)_{10} = (0000 \ 0000 \ 0000)_{BCD}$$

4. 8421 BCD to Decimal Conversion

- Divide the given BCD code into groups of 4 bits (starting from left).
- Convert each 4-bit group to its decimal digit.

Example 1: Convert $(0110 \ 1001)_{BCD}$ to decimal.

$$0110 \rightarrow 6 \quad 1001 \rightarrow 9$$

$$\therefore (0110 \ 1001)_{BCD} = (69)_{10}$$

Example 2: Convert $(0001 \ 0101 \ 0011)_{BCD}$ to decimal.

$$0001 \rightarrow 1 \quad 0101 \rightarrow 5 \quad 0011 \rightarrow 3$$

$$\therefore (0001 \ 0101 \ 0011)_{BCD} = (153)_{10}$$

Example 3: Convert $(1000 \ 0101)_{BCD}$ to decimal.

$$1000 \rightarrow 8 \quad 0101 \rightarrow 5$$

$$\therefore (1000 \ 0101)_{BCD} = (85)_{10}$$

5. Addition using 8421 BCD

Step 1: Add the two BCD numbers (as binary).

Step 2: If the sum of any 4-bit group is ≤ 1001 , it is valid.

Step 3: If the sum is > 1001 or if there is a carry, add 0110 (6) to that 4-bit group.

Step 4: Continue for next group if there is a carry.

Example: Add $(45)_{10}$ and $(37)_{10}$ using 8421 BCD.

$$\begin{array}{r} 45 \rightarrow 0100 \ 0101 \\ 37 \rightarrow 0011 \ 0111 \\ \hline 0111 \ 1100 \text{ (initial sum)} \end{array}$$

$$\therefore \text{Result} = 1000 \ 0010 = (82)_{10}$$

$$\begin{array}{l} \text{Lower nibble} = 1100 (> 1001) \\ \text{Add } 0110 \\ \quad 0111 \ 1100 \\ + 0000 \ 0110 \\ \hline = 1000 \ 0010 \end{array}$$

6. Subtraction using 8421 BCD

Step 1: Subtract the BCD numbers (as binary).

Step 2: If a borrow occurs or if the result is not a valid BCD digit (i.e., > 1001), add 0110 to the result.

Step 3: Continue for next group if necessary.

Example: Subtract $(58)_{10}$ from $(73)_{10}$ using 8421 BCD.

$$\begin{array}{r} 73 \rightarrow 0111 \ 0011 \\ 58 \rightarrow 0101 \ 1000 \\ \hline 0001 \ 1011 \text{ (initial result)} \end{array}$$

$$\therefore \text{Result} = 0010 \ 0001 = (21)_{10}$$

$$\begin{array}{l} \text{Lower nibble} = 1011 (> 1001) \\ \text{Add } 0110 \\ \quad 0001 \ 1011 \\ + 0000 \ 0110 \\ \hline = 0010 \ 0001 \end{array}$$

7. Advantages

- Easy to convert from decimal to BCD.
- Simple to display on digital devices (7-segment display).
- Minimizes errors in decimal representation.

8. Disadvantages

- Requires 4 bits for each decimal digit (more bits than pure binary).
- Not suitable for arithmetic operations directly (needs correction).
- 6 codes (1010-1111) are invalid.

"Different codes are used to represent the same data in different ways."

Binary Codes : Excess-3 & Gray Code

"Codes help in error reduction and reliable data transmission."

1. Excess-3 Code (XS-3)

- Excess-3 code is a non-weighted, self-complementing code.
- It is obtained by adding 3 (0011) to each decimal digit and then writing the result in 4-bit binary form.
- It is also called "XS-3 code".
- It is used in digital systems for error detection and in some arithmetic operations.

2. Excess-3 Code Table

Decimal Digit (D)	Binary (BCD) 8 4 2 1	Add 3 (0011)	Excess-3 Code 8 4 2 1
0	0000	+0011	0011
1	0001	+0011	0100
2	0010	+0011	0101
3	0011	+0011	0110
4	0100	+0011	0111
5	0101	+0011	1000
6	0110	+0011	1001
7	0111	+0011	1010
8	1000	+0011	1011
9	1001	+0011	1100

Key Points :

- Add 3 (0011) to each decimal digit.
- Each digit is represented by 4 bits.
- It is self-complementing code.
- Easy error detection.
- Used in digital devices and calculators.

Note :

Excess-3 code for a digit = (Decimal digit + 3) in 4-bit binary form.

3. Examples (Decimal to Excess-3)

Example 1 : Convert (5)₁₀ to Excess-3.

$$5 = 0101$$

$$\begin{array}{r} \text{Add } 3 = 0011 \\ 0101 \\ \hline 1000 \end{array}$$

∴ Excess-3 code of 5 = 1000

Example 2 : Convert (9)₁₀ to Excess-3.

$$9 = 1001$$

$$\begin{array}{r} \text{Add } 3 = 0011 \\ 1001 \\ \hline 1100 \end{array}$$

∴ Excess-3 code of 9 = 1100

Example 3 : Convert (247)₁₀ to Excess-3.

$$2 = 010 \rightarrow 0101$$

$$4 = 100 \rightarrow 0111$$

$$7 = 111 \rightarrow 1010$$

∴ (247)₁₀ = 0101 0111 1010 (XS-3)

1. Gray Code (Reflected Binary Code)

- Gray code is a non-weighted binary code in which two successive codes differ by only one bit.
- It is also called "Reflected Binary Code".
- It is used in error control, position encoders, and digital communication systems.
- Advantage :** Minimizes errors during data transmission.

2. 4-bit Gray Code Table

Decimal	Binary Code (8 4 2 1)	Gray Code (G ₃ G ₂ G ₁ G ₀)
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Key Points (Gray Code)

- Two successive codes differ by only one bit.
- Reduces chances of error.
- Used in position encoders, A/D converters, and digital communication.
- It is a non-weighted code.
- Conversion involves XOR operation.

3. Method to Convert Binary to Gray Code

Step 1 : Copy the MSB (leftmost bit) as it is.

Step 2 : For each next bit, perform XOR of the current binary bit and the previous binary bit.

Example : Convert (1011)₂ to Gray code.

$$\begin{array}{cccc} \text{Binary : } & 1 & 0 & 1 & 1 & (\text{MSB same}) \\ & \downarrow & \downarrow & \downarrow & \downarrow & \\ & & & & & (1 \oplus 0 = 1) \\ & & & & & (0 \oplus 1 = 1) \\ & & & & & (1 \oplus 1 = 0) \\ \text{Gray : } & 1 & 1 & 1 & 0 & \end{array}$$

$$\therefore (1011)_2 = (1110)_\text{Gray}$$

4. Method to Convert Gray Code to Binary

Step 1 : Copy the MSB (leftmost bit) as it is.

Step 2 : For each next bit, perform XOR of the previous binary bit and the current Gray bit.

Example : Convert (1101)_{Gray} to Binary.

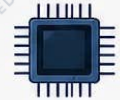
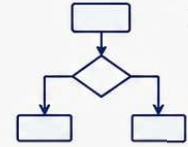
$$\begin{array}{cccc} \text{Gray : } & 1 & 1 & 0 & 1 & (\text{MSB same}) \\ & \downarrow & \downarrow & \downarrow & \downarrow & \\ & & & & & (1 \oplus 1 = 0) \\ & & & & & (0 \oplus 0 = 0) \\ & & & & & (0 \oplus 1 = 1) \\ \text{Binary : } & 1 & 0 & 0 & 1 & \end{array}$$

$$\therefore (1101)_\text{Gray} = (1001)_2$$

Applications

- Rotary encoders
- Digital communication
- Error control
- A/D and D/A converters
- Position sensing devices

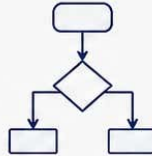
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



"Logic gates are the basic building blocks of digital systems."

Logic Gates : Definition and Truth Tables

"A small gate with a big role in the digital world."

1. Introduction

- Logic gates are electronic circuits that perform a logical operation on one or more binary inputs to produce a single binary output.
- The inputs and outputs are in binary form (0 or 1).
- Logic gates are the fundamental building blocks of digital electronics and are used in computers, calculators, communication systems, etc.

2. Symbols Used

Name	Symbol
Input line	—A
Output line	—→Y
NOT (Inversion bubble)	○
Intersection (no connection)	+
Connection (with dot)	•

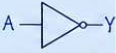
3. Logic Gate Types

There are 7 basic logic gates :

1. NOT Gate (Inverter)
2. AND Gate
3. OR Gate
4. NAND Gate
5. NOR Gate
6. EX-OR Gate
7. EX-NOR Gate

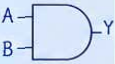
4. NOT Gate (Inverter)

- It gives the complement of the input.
If input is 0, output is 1 and
- if input is 1, output is 0.

Symbol	Boolean Expression	Truth Table	
	$Y = \bar{A}$ (NOT A)	A	Y
		0	1
		1	0

5. AND Gate

- Output is 1 only when all inputs are 1.
- It performs logical multiplication.

Symbol	Boolean Expression	Truth Table		
	$Y = A \cdot B$ (A AND B)	A	B	Y
		0	0	0
		0	1	0
		1	0	0
		1	1	1

6. OR Gate

- Output is 1 when at least one input is 1.
- It performs logical addition.

Symbol	Boolean Expression	Truth Table		
	$Y = A + B$ (A OR B)	A	B	Y
		0	0	0
		0	1	1
		1	0	1
		1	1	1

7. NAND Gate

- It is a combination of AND gate followed by NOT gate.
- Output is 0 only when all inputs are 1.

Symbol	Boolean Expression	Truth Table		
	$Y = \overline{A \cdot B}$ (NAND)	A	B	Y
		0	0	1
		0	1	1
		1	0	1
		1	1	0

8. NOR Gate

- It is a combination of OR gate followed by NOT gate.
- Output is 1 only when all inputs are 0.

Symbol	Boolean Expression	Truth Table		
	$Y = \overline{A + B}$ (NOR)	A	B	Y
		0	0	1
		0	1	0
		1	0	0
		1	1	0

9. EX-OR Gate (Exclusive OR)

- Output is 1 when inputs are different.
- Output is 0 when inputs are same.

Symbol	Boolean Expression	Truth Table		
	$Y = A \oplus B$ (EX-OR)	A	B	Y
		0	0	0
		0	1	1
		1	0	1
		1	1	0

10. EX-NOR Gate (Exclusive NOR)

- Output is 1 when inputs are same.
- Output is 0 when inputs are different.

Symbol	Boolean Expression	Truth Table		
	$Y = \overline{A \oplus B}$ (EX-NOR)	A	B	Y
		0	0	1
		0	1	0
		1	0	0
		1	1	1

Key Points

- Inputs and outputs are in binary form (0 or 1).
- NOT gate has one input, all other basic gates have two inputs.
- NAND and NOR are called Universal Gates (because any logic gate can be implemented using only NAND or only NOR gates).
- EX-OR is used for error detection.
- EX-NOR is used for equality checking (output is 1 when both inputs are same).

Applications

- Computers and processors
- Arithmetic and logic circuits
- Memory devices
- Communication systems
- Control systems
- Digital clocks
- Calculators

"Logic gates make decisions in digital systems."

"NOT gate gives the complement of the input."

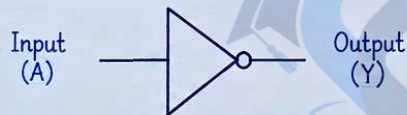
NOT Gate (Inverter)

"It changes 0 to 1 and 1 to 0."

1. Definition

- A NOT gate, also called an Inverter, is a basic logic gate that produces the complement of the input.
- It has only one input and one output.
- If the input is 0, the output is 1, and if the input is 1, the output is 0.
- The output is always the opposite (inverted) logic level of the input.

2. Symbol



NOT gate symbol
(with inversion bubble)

3. Boolean Expression

$$Y = \bar{A}$$

- The output Y is the complement of input A.
- It is read as "Y equals NOT A" or "Y equals A bar".
- Mathematically, if input is A, then output is $Y = \bar{A}$.

4. Truth Table

Input (A)	Output ($Y = \bar{A}$)
0	1
1	0

5. Logic Diagram / Working

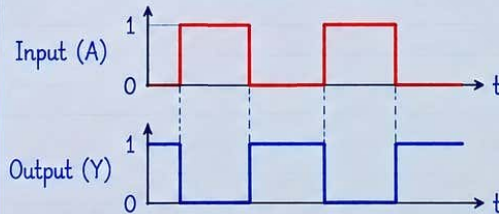


When input is 0,
output is 1.



When input is 1,
output is 0.

6. Waveform Representation



Output waveform is always the inverse of input waveform.

7. Important Points

- A NOT gate has only one input.
- It is also called an Inverter.
- The output is the complement of the input.
- It is represented by a triangle with a small circle at the output (inversion bubble).
- Boolean expression: $Y = \bar{A}$
- If input is 0 → output is 1.
- If input is 1 → output is 0.
- It is a fundamental building block for other logic circuits.

8. Real Life Analogy

- It works like a light switch:
 - If the switch is OFF (0) → light is ON (1)
 - If the switch is ON (1) → light is OFF (0)

9. Applications

- Used in digital circuits to invert a signal.
- Used in memory circuits (to store complement).
- Used in arithmetic and logic operations.
- Used in combination with other gates to design complex logic circuits.

"NOT gate is simple, but it plays an important role in digital electronics."

"OR gate gives 1 when at least one input is 1."

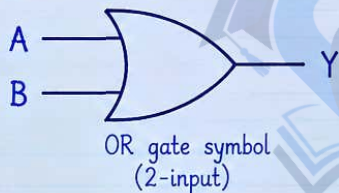
OR Gate

"It performs logical addition and is used to combine inputs."

1. Definition

- An OR gate is a basic logic gate that produces an output of 1 when at least one of the inputs is 1.
- It has two or more inputs and one output.
- If all the inputs are 0, the output is 0.
- If any one or more inputs are 1, the output is 1.
- It performs the logical addition operation.

2. Symbol



3. Boolean Expression

$$Y = \bar{A} + B$$

- The output Y is the logical OR of inputs A and B.
- It is read as "A OR B" or " $A + B$ ".
- The output is 1 if $A = 1$ or $B = 1$ or both are 1.
- The output is 0 only when $A = 0$ and $B = 0$.

4. Truth Table

A	B	$Y = \bar{A} + B$
0	0	0
0	1	1
1	0	1
1	1	1

5. Logic Diagram / Working

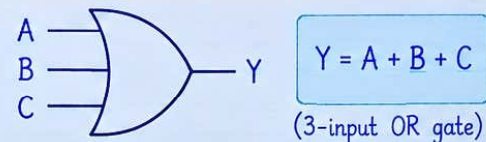
Case 1 : $A = 0, B = 0 \rightarrow Y = 0$
(Both inputs 0
→ output 0)

Case 2 : $A = 0, B = 1 \rightarrow Y = 1$
(At least one input 1,
→ output 1)

Case 3 : $A = 1, B = 0 \rightarrow Y = 1$
(At least one input 1,
→ output 1)

Case 4 : $A = 1, B = 1 \rightarrow Y = 1$
(Both inputs 1
→ output 1)

6. Gate with More Inputs



7. Important Points

- Output is 1 when at least one input is 1.
- Output is 0 only when all inputs are 0.
- It performs logical addition.
- It can have two or more inputs.
- Used to combine multiple signals.
- Widely used in digital circuits and control systems.

8. Applications

- Used in alarm systems (if any one sensor detects a fault).
- Used in data selection and control circuits.
- Used in digital communication systems.
- Used in combinational logic design.
- Used to detect the presence of a signal from multiple sources.

"OR gate helps to get an output when any input is active."

"AND gate gives 1 only when all inputs are 1."

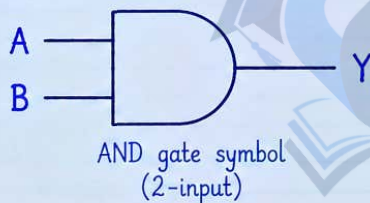
AND Gate

"It performs logical multiplication."

1. Definition

- An AND gate is a basic logic gate that produces an output of 1 only when all the inputs are 1.
- It has two or more inputs and one output.
- If any one input is 0, the output is 0.
- It performs logical multiplication.
- It is widely used in digital circuits, control systems and computers.

2. Symbol



3. Boolean Expression

$$Y = A \cdot B$$

- The output Y is the logical AND of inputs A and B.
- It is read as "A AND B" or " $A \cdot B$ ".
- The output is 1 only if $A = 1$ and $B = 1$.
- Otherwise, the output is 0.

4. Truth Table

A	B	$Y = \bar{A} \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

5. Logic Diagram / Working

Case 1 : $A = 0, B = 0$



Case 2 : $A = 0, B = 1$



Case 3 : $A = 1, B = 0$



Case 4 : $A = 1, B = 1$

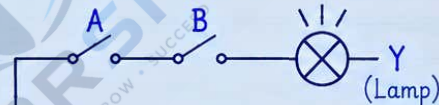


6. Key Points

- Output is 1 only when all inputs are 1.
- Output is 0 if any one (or more) input is 0.
- It performs logical multiplication.
- It can have two or more inputs.
- Boolean expression (2-input) : $Y = \bar{A} \cdot B$
- Used in many digital circuits and decision making systems.

7. Example

Consider two switches A and B connected in series to a lamp. The lamp (output Y) will glow only when both switches are closed ($A = 1$ and $B = 1$).



8. Applications

- Used in computers and processors.
- Used in control systems and automation.
- Used in security systems (e.g., door opens only when two conditions are satisfied).
- Used in arithmetic circuits (e.g., multiplication).
- Used in digital communication systems.

"AND gate ensures that all conditions are true before giving an output."

"NAND gate is a universal gate because any logic function can be implemented using only NAND gates."

NAND Gate

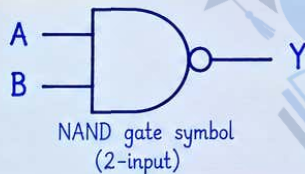
(NOT - AND gate)

"It performs the complement of AND operation. Output is 0 only when all inputs are 1."

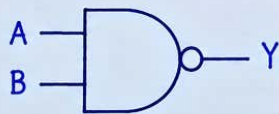
1. Definition

- A NAND gate is a basic logic gate that produces the complement of the AND operation.
- It gives an output 0 only when all inputs are 1, otherwise it gives an output 1.
- It is called a universal gate because any other logic gate (AND, OR, NOT, etc.) can be constructed using only NAND gates.
- It is widely used in digital circuits, computers and control systems.

2. Symbol



6. Logic Diagram / Working



First perform AND operation on inputs A and B, then invert the result.
So, output is the opposite of the AND output.

Y = 1 when (A,B) = (0,0), (0,1), (1,0)
Y = 0 when (A,B) = (1,1)

3. Boolean Expression

$$Y = \overline{A \cdot B}$$

- The output Y is the complement of the AND of inputs A and B.
- It is read as "NOT (A AND B)" or "A NAND B".
- Mathematically,

$$Y = \overline{A \cdot B}$$

4. Truth Table

A	B	$Y = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

5. Logic Operation

- If A = 0 and B = 0 $\rightarrow$ Y = 1
- If A = 0 and B = 1 $\rightarrow$ Y = 1
- If A = 1 and B = 0 $\rightarrow$ Y = 1
- If A = 1 and B = 1 $\rightarrow$ Y = 0

Note: Output is 0 only when both inputs are 1, otherwise output is 1.

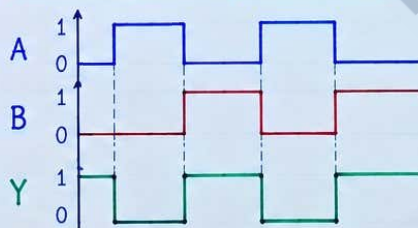
8. Important Points

- NAND gate is a universal gate.
- Output is 0 only when all inputs are 1.
- It is the complement of AND gate.
- Boolean expression : $Y = \overline{A \cdot B}$
- Used in many digital circuits, memory devices and processors.
- Any logic gate (AND, OR, NOT) can be implemented using only NAND gates.
- It is very useful in IC design.

9. Applications

- Used in computers and microprocessors.
- Used in memory circuits (e.g., RAM, flip-flops).
- Used in control systems and automation.
- Used to design other logic gates.
- Widely used in digital electronics and VLSI design.

7. Waveform Representation



"NOR gate gives output 1 only when all inputs are 0."

NOR Gate

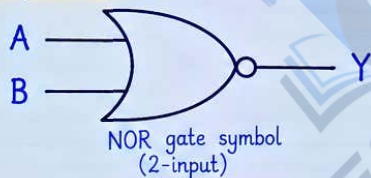
(NOT - OR gate)

"It is the complement of OR gate and is called a universal gate."

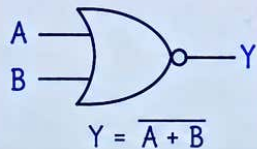
1. Definition

- A NOR gate is a basic logic gate that produces the complement of the OR operation.
- It gives an output 1 only when all inputs are 0, otherwise it gives an output 0.
- It is called a universal gate because any other logic gate (AND, OR, NOT, etc.) can be implemented using only NOR gates.
- It is widely used in digital circuits, computers and control systems.

2. Symbol



6. Logic Diagram / Working



First perform OR operation on inputs A and B, then invert the result. So, output is the opposite of the OR output.

"NOR gate may look simple, but it is powerful because it can build any logic circuit."

3. Boolean Expression

$$Y = \overline{A + B}$$

- The output Y is the complement of the OR of inputs A and B.
- It is read as "NOT (A OR B)" or "A NOR B".
- Mathematically,

$$Y = \overline{A + B}$$

4. Truth Table

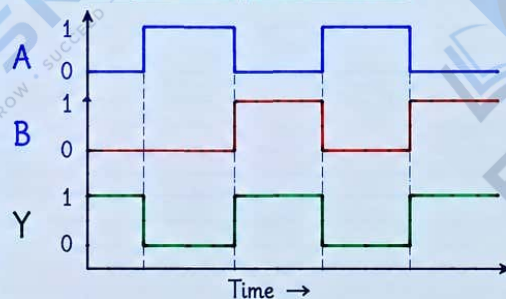
A	B	$Y = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

5. Logic Operation

- If $A = 0$ and $B = 0 \rightarrow Y = 1$
- If $A = 0$ and $B = 1 \rightarrow Y = 0$
- If $A = 1$ and $B = 0 \rightarrow Y = 0$
- If $A = 1$ and $B = 1 \rightarrow Y = 0$

Note : Output is 1 only when both inputs are 0, otherwise output is 0.

7. Waveform Representation



Output waveform is the inverse of the OR operation on inputs A and B.

8. Important Points

- NOR gate is a universal gate.
- Output is 1 only when all inputs are 0.
- It is the complement of OR gate.
- Boolean expression : $Y = \overline{A + B}$
- Can have two or more inputs.
- Used in many digital circuits, memory devices and processors.
- Any logic gate (AND, OR, NOT) can be implemented using only NOR gates.
- It is very useful in VLSI design.

9. Applications

- Used in computers and microprocessors.
- Used in memory circuits (e.g., RAM, flip-flops).
- Used in control systems and automation.
- Used to design other logic gates.
- Widely used in digital electronics and VLSI design.

"EX-OR gives 1 when inputs are different."

EX-OR and EX-NOR Gates

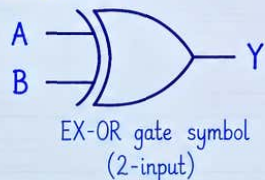
"EX-NOR gives 1 when inputs are the same."

1. EX-OR Gate (Exclusive-OR)

Definition :

- An EX-OR gate is a logic gate that gives an output 1 when the inputs are different.
- It gives an output 0 when the inputs are the same.
- It is also called an Exclusive-OR gate because only one input should be 1, not both.
- It is widely used in error detection, parity checking and digital circuits.

2. Symbol :



3. Boolean Expression :

$$Y = A \oplus B$$

$$Y = \bar{A}B + A\bar{B}$$

4. Truth Table :

A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

5. Working :

- If $A = 0, B = 0 \rightarrow Y = 0$ (inputs same)
- If $A = 0, B = 1 \rightarrow Y = 1$ (inputs different)
- If $A = 1, B = 0 \rightarrow Y = 1$ (inputs different)
- If $A = 1, B = 1 \rightarrow Y = 0$ (inputs same)

6. Applications :

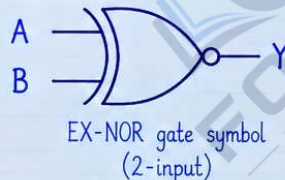
- Used in error detection and parity checking.
- Used in adders and subtractors in digital circuits.
- Used in data comparison.
- Used in communication systems.

1. EX-NOR Gate (Exclusive-NOR)

Definition :

- An EX-NOR gate is a logic gate that gives an output 1 when the inputs are the same.
- It gives an output 0 when the inputs are different.
- It is also called an Exclusive-NOR gate because the inputs must be equal (both 0 or both 1).
- It is widely used in comparison circuits, error detection and control systems.

2. Symbol :



3. Boolean Expression :

$$Y = A \odot B$$

$$Y = \bar{A}\bar{B} + AB$$

4. Truth Table :

A	B	$Y = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

5. Working :

- If $A = 0, B = 0 \rightarrow Y = 1$ (inputs same)
- If $A = 0, B = 1 \rightarrow Y = 0$ (inputs different)
- If $A = 1, B = 0 \rightarrow Y = 0$ (inputs different)
- If $A = 1, B = 1 \rightarrow Y = 1$ (inputs same)

6. Applications :

- Used in equality comparison circuits.
- Used in error detection and parity checking.
- Used in data transmission systems.
- Used in control and security systems.

Key Difference : EX-OR gives 1 when inputs are different, while EX-NOR gives 1 when inputs are the same.

"NAND and NOR are called Universal Gates because any logic function can be implemented using only these gates."

Universal Gates

"They are functionally complete, meaning all other logic gates (AND, OR, NOT, EX-OR, EX-NOR) can be constructed using only NAND or NOR gates."

1. What are Universal Gates?

- NAND and NOR gates are called Universal Gates.
- They can be used to implement any other logic gate (AND, OR, NOT, EX-OR, EX-NOR).
- They are functionally complete, meaning any Boolean expression can be realized using only NAND or only NOR gates.
- They are widely used in digital circuit design, VLSI and computer systems.

2. Universal Gates

- NAND Gate (NOT-AND)**
 - Symbol: AND gate with inversion bubble at output.
 - Output is **0** only when all inputs are 1.
 - Denoted as : $A \uparrow B$ or $\overline{A \cdot B}$
- NOR Gate (NOT-OR)**
 - Symbol: OR gate with inversion bubble at output.
 - Output is **1** only when all inputs are 0.
 - Denoted as : $A \downarrow B$ or $\overline{A + B}$

3. Truth Tables

NAND Gate		
A	B	$Y = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate		
A	B	$Y = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

4. Realization of Basic Gates using NAND Gate

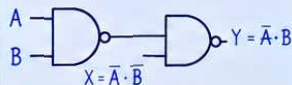
a) NOT Gate using NAND

Connect both inputs together.



$$Y = A \text{ NAND } A = \overline{A}$$

b) AND Gate using NAND



$$Y = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B) = A \cdot B$$

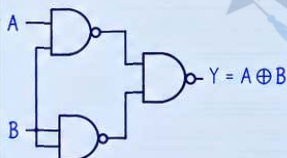
c) OR Gate using NAND

Use De Morgan's law : $\overline{A + B} = \overline{A} \cdot \overline{B}$



$$Y = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$$

d) EX-OR Gate using NAND

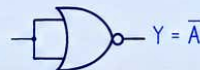


$$Y = (A \text{ NAND } (A \text{ NAND } B)) \text{ NAND } (B \text{ NAND } (A \text{ NAND } B))$$

5. Realization of Basic Gates using NOR Gate

a) NOT Gate using NOR

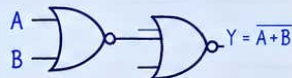
Connect both inputs together.



$$Y = A \text{ NOR } A = \overline{A}$$

b) OR Gate using NOR

Use De Morgan's law : $A + B = \overline{\overline{A} \cdot \overline{B}}$



$$Y = (A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B) = A + B$$

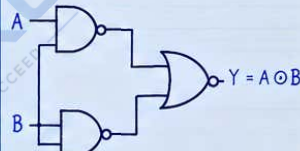
c) AND Gate using NOR

Use De Morgan's law : $\overline{A \cdot B} = \overline{A} + \overline{B}$



$$Y = (A \text{ NOR } A) \text{ NOR } (B \text{ NOR } B)$$

d) EX-NOR Gate using NOR



$$Y = (A \text{ NOR } (A \text{ NOR } B)) \text{ NOR } (B \text{ NOR } (A \text{ NOR } B))$$

6. Key Points

- NAND and NOR are called Universal Gates.
- Any logic gate (NOT, AND, OR, EX-OR, EX-NOR) can be constructed using only NAND gates or only NOR gates.
- They are functionally complete.
- NAND is 0 only when all inputs are 1.
- NOR is 1 only when all inputs are 0.
- Universal gates are widely used in VLSI, computer architecture, and digital system design.

7. Conclusion

- NAND and NOR gates are very important because they can be used to implement any other logic gate and any Boolean expression. Hence, they are called Universal Gates.

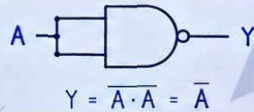
"With just NAND or just NOR, we can build all digital logic."

Designing of Basic Gates using NAND and NOR Gates

Designing Basic Gates using NAND Gates

- **NAND** is a universal gate.
- Any basic logic gate (NOT, AND, OR) can be implemented using only NAND gates.
- NAND gate output : $Y = A \cdot B$
- We can obtain other gates by suitable interconnections of NAND gates.

1. NOT gate using NAND



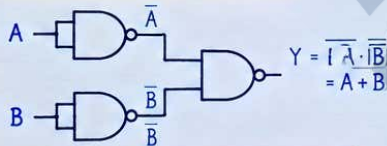
Connect both inputs of NAND to the same input.

2. AND gate using NAND



Two NAND gates are used.

3. OR gate using NAND



First invert both inputs using NAND, then apply NAND.

4. NAND gate (for reference)



This is the basic NAND gate.

Designing Basic Gates using NOR Gates

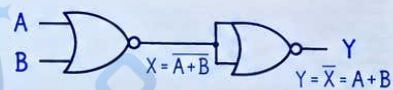
- **NOR** is a universal gate.
- Any basic logic gate (NOT, AND, OR) can be implemented using only NOR gates.
- NOR gate output : $Y = \overline{A + B}$
- We can obtain other gates by suitable interconnections of NOR gates.

1. NOT gate using NOR



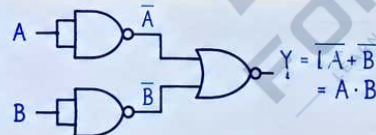
Connect both inputs of NOR to the same input.

2. OR gate using NOR



Two NOR gates are used.

3. AND gate using NOR



First invert both inputs using NOR, then apply NOR.

4. NOR gate (for reference)



This is the basic NOR gate.

Key Points :

- NAND and NOR gates are called Universal Gates.
- Using only NAND gates or only NOR gates, we can implement NOT, AND, OR and any other logic circuit.
- They are functionally complete.
- Universal gates are widely used in digital electronics, VLSI design and computer architecture.

Conclusion :

Any basic logic gate (NOT, AND, OR) can be designed using only NAND gates or only NOR gates by properly connecting the gates. Therefore, NAND and NOR are known as universal gates.

"A Half Adder adds two single binary bits and produces a Sum and a Carry."

Half Adder and Full Adder

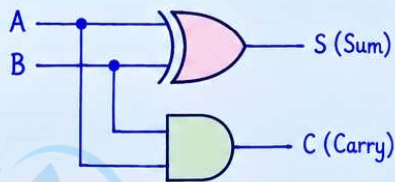
"A Full Adder adds three binary bits (A, B and C_{in}) and produces a Sum and a Carry."

1. Half Adder

Definition :

- A Half Adder is a combinational logic circuit that adds two single binary bits (A and B).
- It produces two outputs : Sum (S) and Carry (C).
- It does not consider any carry input from the previous stage.
- It is the simplest form of an adder.

2. Logic Diagram



3. Boolean Expressions

$$S = A \oplus B$$

$$C = A \cdot B$$

- Sum is the XOR of A and B.
- Carry is the AND of A and B.

4. Truth Table (Half Adder)

A	B	S (Sum)	C (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

5. Working

- If $A = 0, B = 0 \rightarrow S = 0, C = 0$
- If $A = 0, B = 1 \rightarrow S = 1, C = 0$
- If $A = 1, B = 0 \rightarrow S = 1, C = 0$
- If $A = 1, B = 1 \rightarrow S = 0, C = 1$

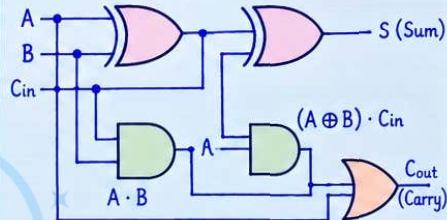
Note : Carry is generated only when both inputs are 1.

4. Full Adder

Definition :

- A Full Adder is a combinational logic circuit that adds three binary bits (A, B and C_{in}).
- It produces two outputs : Sum (S) and Carry (C_{out}).
- It takes a carry input (C_{in}) from the previous stage.
- It is widely used in multi-bit binary addition (adders in computers and arithmetic units).

5. Logic Diagram



6. Boolean Expressions

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + (A \oplus B) \cdot C_{in}$$

7. Alternate Expression for Carry

$$C_{out} = A \cdot B + B \cdot C_{in} + A \cdot C_{in}$$

(Majority function)

8. Truth Table (Full Adder)

A	B	C_{in}	S (Sum)	C_{out} (Carry)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

9. Working

- Sum is 1 when an odd number of inputs are 1.
- Carry is 1 when at least two inputs are 1.
- It can be implemented using two half adders and an OR gate.

Note : A full adder can be constructed using two half adders.

Applications :

Used in arithmetic and logic units (ALU), binary adders, digital calculators, computers and other digital systems.

"A 4-bit binary full adder adds two 4-bit binary numbers and produces a 4-bit Sum and a Carry out."

4-Bit Binary Full Adder

"It is made by cascading four 1-bit full adders."

1. Introduction

- A 4-bit binary full adder is a combinational logic circuit that adds two 4-bit binary numbers ($A_3 A_2 A_1 A_0$ and $B_3 B_2 B_1 B_0$) and produces a 4-bit sum ($S_3 S_2 S_1 S_0$) and a carry out (C_{out}).
- It is formed by cascading four 1-bit full adders.
- The carry output of each stage is connected to the carry input of the next stage.
- It is used in ALU, processors and other digital systems.

2. 1-Bit Full Adder (Basic Unit)

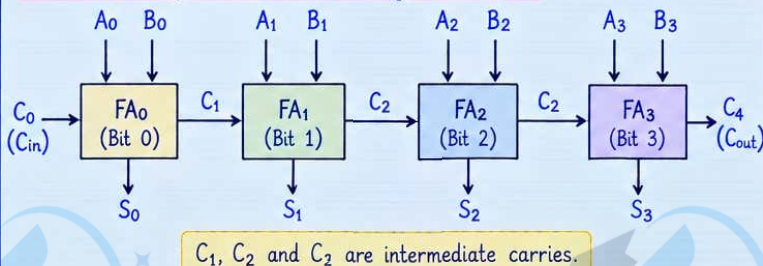
- A 1-bit full adder adds three inputs A, B and C_{in} and produces a Sum (S) and Carry (C_{out}).

$$S = A \oplus B \oplus C_{in}$$

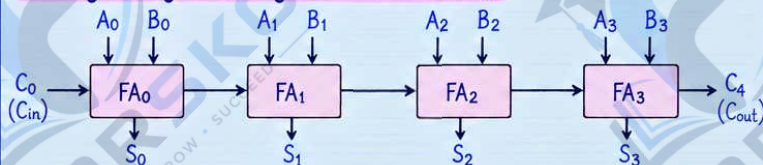
$$C_{out} = A \cdot B + (A \oplus B) \cdot C_{in}$$



3. Block Diagram of 4-Bit Binary Full Adder



4. Logic Diagram (using 1-Bit Full Adders)



5. Truth Table (1-Bit Full Adder)

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

6. Example

Add two 4-bit numbers :

$$A = 1101 \text{ (13)}$$

$$B = 0111 \text{ (7)}$$

$$\begin{array}{r} 1101 \text{ (A)} \\ + 0111 \text{ (B)} \\ \hline 10100 \text{ (Result)} \end{array}$$

$$C_{out} = 1, \text{ Sum} = 0100 \text{ (4)}$$

$$\text{So, } 13 + 7 = 20 \text{ (10100 in binary)}$$

7. Working

- The least significant bits (A_0, B_0) and initial carry ($C_0 = 0$) are added using the first full adder to obtain S_0 and C_1 .
- The carry C_1 is passed to the next stage along with A_1 and B_1 .
- This process continues for all four stages.
- Finally, we get 4-bit sum ($S_3 S_2 S_1 S_0$) and a carry out (C_4).
- Each stage is a 1-bit full adder, so the circuit can be easily extended to n-bits by cascading n full adders.

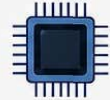
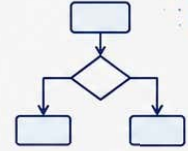
8. Applications

- Used in Arithmetic Logic Unit (ALU).
- Used in microprocessors and microcontrollers.
- Used in digital calculators.
- Used in data processing and signal processing systems.
- Used in computers and other digital electronics.

Key Points :

- A 4-bit full adder adds two 4-bit numbers and gives a 4-bit sum and a carry.
- It is made by cascading four 1-bit full adders.
- Each full adder adds A, B and C_{in} .
- C_{out} is generated from the most significant stage.
- It can be extended to n-bit adders.

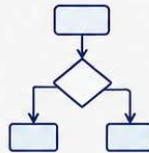
1010101010
0101010101
1010101010
0101010101



Unit - III



10110
01001
10101



"Boolean Algebra is the mathematical foundation of digital logic design."

Boolean Algebra : Boolean Laws

"These laws help to simplify and manipulate Boolean expressions."

1. Introduction

- Boolean Algebra is a branch of algebra used to analyze and design digital logic circuits.
- It was developed by George Boole in 1854.
- It uses binary variables (0 and 1) and three basic operations : AND ($\cdot$), OR ($+$) and NOT ($'$).
- Boolean laws are rules used to simplify Boolean expressions.

2. Basic Operators

Operation	Symbol	Expression
AND	$\cdot$	$A \cdot B$
OR	$+$	$A + B$
NOT	$'$	$\bar{A}$ (or A')

3. Fundamental Laws

(i) Commutative Law

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

(ii) Associative Law

$$(A + B) + C = A + (B + C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

(iii) Distributive Law

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

(iv) Identity Law

$$A + 0 = A$$

$$A \cdot 1 = A$$

(v) Null (Domination) Law

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

(vi) Idempotent Law

$$A + A = A$$

$$A \cdot A = A$$

(vii) Complement Law

$$A + \bar{A} = 1$$

$$A \cdot \bar{A} = 0$$

(viii) Involution Law

$$(\bar{A})' = A$$

(ix) Absorption Law

$$A + A \cdot B = A$$

$$A \cdot (A + B) = A$$

(x) De Morgan's Law

$$(A + B)' = \bar{A} \cdot \bar{B} \quad (A \cdot B)' = \bar{A} + \bar{B}$$

(xi) Duality Principle

- If we interchange $+$ and $\cdot$, and 0 and 1 in any Boolean expression, we get its dual expression.
- Example : $A + 0 = A \Leftrightarrow A \cdot 1 = A$

4. Summary Table

Law Name	Expression
Commutative	$A + B = B + A$, $A \cdot B = B \cdot A$
Associative	$(A + B) + C = A + (B + C)$, $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$, $A + (B \cdot C) = (A + B) \cdot (A + C)$
Identity	$A + 0 = A$, $A \cdot 1 = A$
Null (Domination)	$A + 1 = 1$, $A \cdot 0 = 0$
Idempotent	$A + A = A$, $A \cdot A = A$
Complement	$A + \bar{A} = 1$, $A \cdot \bar{A} = 0$
Involution	$(\bar{A})' = A$
Absorption	$A + A \cdot B = A$, $A \cdot (A + B) = A$
De Morgan's	$(A + B)' = \bar{A} \cdot \bar{B}$, $(A \cdot B)' = \bar{A} + \bar{B}$

Key Takeaway :

Boolean laws are used to simplify logic expressions and are very important in digital circuit design, minimization of logic functions and Boolean algebraic manipulation.

"De-Morgan's Theorem helps us convert between AND and OR operations using complementation."

De-Morgan's Theorem

"The complement of a product is equal to the sum of the complements and the complement of a sum is equal to the product of the complements."

1. Introduction

- De-Morgan's Theorem is a fundamental law in Boolean Algebra.
- It gives a relationship between complement, AND and OR operations.
- It is widely used in digital logic design to simplify logic expressions and to implement circuits using only NAND or NOR gates.

2. Theorems

(i) First De-Morgan's Theorem

$$\overline{(A \cdot B)} = \bar{A} + \bar{B}$$

"The complement of a product is equal to the sum of the complements."

(ii) Second De-Morgan's Theorem

$$\overline{(A + B)} = \bar{A} \cdot \bar{B}$$

"The complement of a sum is equal to the product of the complements."

3. Verification using Truth Tables

We can verify both the theorems using truth tables.

(i) Truth Table for $\overline{(A \cdot B)} = \bar{A} + \bar{B}$

A	B	$A \cdot B$	$\overline{(A \cdot B)}$	$\bar{A}$	$\bar{B}$	$\bar{A} + \bar{B}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

$$\text{Thus, } \overline{(A \cdot B)} = \bar{A} + \bar{B} \quad (\text{Verified})$$

(ii) Truth Table for $\overline{(A + B)} = \bar{A} \cdot \bar{B}$

A	B	$A + B$	$\overline{(A + B)}$	$\bar{A}$	$\bar{B}$	$\bar{A} \cdot \bar{B}$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

$$\text{Thus, } \overline{(A + B)} = \bar{A} \cdot \bar{B} \quad (\text{Verified})$$

5. Examples

(i) Simplify $\overline{(A \cdot B \cdot C)}$

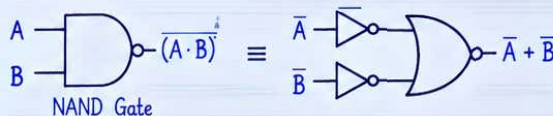
$$\begin{aligned} \overline{(A \cdot B \cdot C)} &= \overline{(A \cdot B)} + \bar{C} \\ &= (\bar{A} + \bar{B}) + \bar{C} \\ &= \bar{A} + \bar{B} + \bar{C} \end{aligned}$$

(ii) Simplify $\overline{(A + B + C)}$

$$\begin{aligned} \overline{(A + B + C)} &= \overline{(A + B)} \cdot \bar{C} \\ &= (\bar{A} \cdot \bar{B}) \cdot \bar{C} \\ &= \bar{A} \cdot \bar{B} \cdot \bar{C} \end{aligned}$$

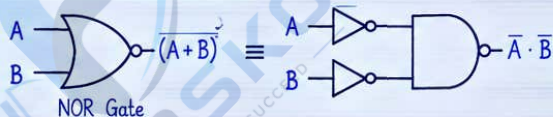
4. Logic Gate Representation

(i) $\overline{(A \cdot B)} = \bar{A} + \bar{B}$



A NAND gate is equivalent to an OR gate with inverted inputs.

(ii) $\overline{(A + B)} = \bar{A} \cdot \bar{B}$



A NOR gate is equivalent to an AND gate with inverted inputs.

6. Key Points

- De-Morgan's Theorem is very useful in simplifying Boolean expressions.
- It helps in converting between AND and OR operations using complementation.
- It is used in the design of logic circuits with NAND and NOR gates.
- The theorems are valid for any number of variables.

Conclusion :

De-Morgan's Theorem provides a simple and powerful way to manipulate Boolean expressions and is essential in digital logic design.

"Simplification means reducing a Boolean expression to its simplest form using the rules and laws of Boolean Algebra."

Simplification of Boolean Equations using Boolean Algebra

"A simplified expression has fewer terms and literals, making the logic circuit easier, cheaper and faster to implement."

1. Important Boolean Laws (for simplification)

(i) Commutative Law

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

(ii) Associative Law

$$(A + B) + C = A + (B + C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

(iii) Distributive Law

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

(iv) Identity Law

$$A + 0 = A$$

$$A \cdot 1 = A$$

(v) Null (Domination) Law

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

(vi) Idempotent Law

$$A + A = A$$

$$A \cdot A = A$$

(vii) Complement Law

$$A + \bar{A} = 1$$

$$A \cdot \bar{A} = 0$$

(viii) Involution Law

$$(\bar{A})' = A$$

(ix) Absorption Law

$$A + A \cdot B = A$$

$$A \cdot (A + B) = A$$

(x) De Morgan's Theorem

$$(A + \bar{B}) = \bar{A} \cdot \bar{B}$$

$$(A \cdot \bar{B}) = \bar{A} + \bar{B}$$

2. Examples of Simplification

Example 1: Simplify $A + A \cdot B$

Solution:

$$A + A \cdot B$$

$$= A \cdot (1 + B) \quad [\text{Distributive}]$$

$$= A \cdot 1 \quad [\text{Since } 1 + B = 1]$$

$$= A \quad [\text{Identity}]$$

$$\therefore A + A \cdot B = A$$

Example 2: Simplify $A \cdot (A + B)$

Solution:

$$A \cdot (A + B)$$

$$= A \cdot A + A \cdot B \quad [\text{Distributive}]$$

$$= A + A \cdot B \quad [A \cdot A = A]$$

$$= A \quad [\text{Absorption}]$$

$$\therefore A \cdot (A + B) = A$$

Example 3: Simplify $A + \bar{A} \cdot B$

Solution:

$$A + \bar{A} \cdot B$$

$$= (A + \bar{A}) \cdot (A + B) \quad [\text{Distributive}]$$

$$= 1 \cdot (A + B) \quad [A + \bar{A} = 1]$$

$$= A + B \quad [\text{Identity}]$$

$$\therefore A + \bar{A} \cdot B = A + B$$

Example 4: Simplify $A \cdot \bar{A} + B$

Solution:

$$A \cdot \bar{A} + B$$

$$= 0 + B \quad [A \cdot \bar{A} = 0]$$

$$= B \quad [\text{Identity}]$$

$$\therefore A \cdot \bar{A} + B = B$$

Example 5: Simplify $(A + B) \cdot (A + \bar{B})$

Solution:

$$(A + B) \cdot (A + \bar{B})$$

$$= A + B \cdot \bar{B} \quad [(X + Y)(X + Z) = X + YZ]$$

$$= A + 0 \quad [B \cdot \bar{B} = 0]$$

$$= A \quad [\text{Identity}]$$

$$\therefore (A + B) \cdot (A + \bar{B}) = A$$

Example 6: Simplify $A \cdot B + A \cdot \bar{B}$

Solution:

$$A \cdot B + A \cdot \bar{B}$$

$$= A \cdot (B + \bar{B}) \quad [\text{Distributive}]$$

$$= A \cdot 1 \quad [B + \bar{B} = 1]$$

$$= A \quad [\text{Identity}]$$

$$\therefore A \cdot B + A \cdot \bar{B} = A$$

Example 7: Simplify $\bar{A} + A \cdot B$

Solution:

$$\bar{A} + A \cdot B$$

$$= (\bar{A} + A) \cdot (\bar{A} + B) \quad [\text{Distributive}]$$

$$= 1 \cdot (\bar{A} + B) \quad [\bar{A} + A = 1]$$

$$= \bar{A} + B \quad [\text{Identity}]$$

$$\therefore \bar{A} + A \cdot B = \bar{A} + B$$

Example 8: Simplify $(A \cdot \bar{B})$

Solution:

$$(A \cdot \bar{B})$$

$$= \bar{A} + \bar{B} \quad [\text{De Morgan's}]$$

$$\therefore (A \cdot \bar{B}) = \bar{A} + \bar{B}$$

Example 9: Simplify $A + \bar{A} \cdot \bar{B} + A \cdot \bar{B}$

Solution:

$$A + \bar{A} \cdot \bar{B} + A \cdot \bar{B}$$

$$= A + B \cdot (\bar{A} + A) \quad [\bar{A} \cdot B + A \cdot \bar{B} = B \cdot (\bar{A} + A)]$$

$$= A + B \cdot 1 \quad [\bar{A} + A = 1]$$

$$= A + B \quad [\text{Identity}]$$

$$\therefore A + \bar{A} \cdot \bar{B} + A \cdot \bar{B} = A + B$$

Key Points:

- Use Boolean laws to simplify step by step.
- Look for common terms and apply absorption or distributive law.
- The simplified expression should be equivalent to the original expression.
- Simplification helps in reducing the number of gates in a digital circuit.

Conclusion:

Boolean algebra provides a systematic way to simplify equations, making digital circuits more efficient and easier to design.

"Any Boolean expression can be written as a sum of minterms or a product of maxterms."

Fundamental Products & Sum Terms

"Minterms represent the fundamental products and maxterms represent the fundamental sum terms."

1. Fundamental Product Term (Minterm)

- A fundamental product term is a product (AND) of all the variables in the expression, where each variable appears either in complemented form or uncomplemented form but not both.
- It is called a minterm because it produces a value 1 for only one unique combination of input variables and 0 for all other combinations.
- For n variables, the number of minterms is 2^n .

Example : (for 3 variables A, B, C)

$$\begin{aligned} m_0 &= \bar{A} \bar{B} \bar{C} \\ m_1 &= \bar{A} \bar{B} C \\ m_2 &= \bar{A} B \bar{C} \\ m_3 &= \bar{A} B C \\ m_4 &= A \bar{B} \bar{C} \\ m_5 &= A \bar{B} C \\ m_6 &= A B \bar{C} \\ m_7 &= A B C \end{aligned}$$

Each minterm corresponds to a unique row of the truth table where the output is 1.

2. Fundamental Sum Term (Maxterm)

- A fundamental sum term is a sum (OR) of all the variables in the expression, where each variable appears either in complemented form or uncomplemented form but not both.
- It is called a maxterm because it produces a value 0 for only one unique combination of input variables and 1 for all other combinations.
- For n variables, the number of maxterms is 2^n .

Example : (for 3 variables A, B, C)

$$\begin{aligned} M_0 &= (A + B + C) \\ M_1 &= (A + B + \bar{C}) \\ M_2 &= (A + \bar{B} + C) \\ M_3 &= (A + \bar{B} + \bar{C}) \\ M_4 &= (\bar{A} + B + C) \\ M_5 &= (\bar{A} + B + \bar{C}) \\ M_6 &= (\bar{A} + \bar{B} + C) \\ M_7 &= (\bar{A} + \bar{B} + \bar{C}) \end{aligned}$$

Each maxterm corresponds to a unique row of the truth table where the output is 0.

3. Truth Table with Minterms and Maxterms (for 3 variables)

A	B	C	Minterm (m_i)	Maxterm (M_i)
0	0	0	$m_0 = \bar{A} \bar{B} \bar{C}$	$M_0 = (A + B + C)$
0	0	1	$m_1 = \bar{A} \bar{B} C$	$M_1 = (A + B + \bar{C})$
0	1	0	$m_2 = \bar{A} B \bar{C}$	$M_2 = (A + \bar{B} + C)$
0	1	1	$m_3 = \bar{A} B C$	$M_3 = (A + \bar{B} + \bar{C})$
1	0	0	$m_4 = A \bar{B} \bar{C}$	$M_4 = (\bar{A} + B + C)$
1	0	1	$m_5 = A \bar{B} C$	$M_5 = (\bar{A} + B + \bar{C})$
1	1	0	$m_6 = A B \bar{C}$	$M_6 = (\bar{A} + \bar{B} + C)$
1	1	1	$m_7 = A B C$	$M_7 = (\bar{A} + \bar{B} + \bar{C})$

4. Standard Forms Using Fundamental Terms

(i) Sum of Minterms (Canonical SOP)

- A Boolean function can be written as a sum (OR) of the minterms for which the function is 1.
- Notation : $F(A, B, C) = \sum m(i)$
- Example : If $F = 1$ for m_1, m_3, m_5

$$\begin{aligned} F(A, B, C) &= \sum m(1, 3, 5) \\ &= \bar{A} \bar{B} C + \bar{A} B C + A \bar{B} C \end{aligned}$$

(ii) Product of Maxterms (Canonical POS)

- A Boolean function can be written as a product (AND) of the maxterms for which the function is 0.
- Notation : $F(A, B, C) = \prod M(i)$
- Example : If $F = 0$ for M_0, M_2, M_6

$$\begin{aligned} F(A, B, C) &= \prod M(0, 2, 6) \\ &= (A + B + C) (A + \bar{B} + C) (\bar{A} + \bar{B} + C) \end{aligned}$$

5. Important Points

- Fundamental product terms are called minterms.
- Fundamental sum terms are called maxterms.
- Each minterm has all variables in product form.
- Each maxterm has all variables in sum form.
- For n variables, there are 2^n minterms and 2^n maxterms.
- Used to express Boolean functions in canonical forms (SOP and POS).

6. Conclusion

- Fundamental product terms (minterms) and fundamental sum terms (maxterms) provide a standard and systematic way to represent Boolean functions.
- They are very useful in simplifying and designing digital logic circuits.
- Every Boolean function can be expressed either as a sum of minterms or a product of maxterms.

"Minterms give 1 for a unique combination, maxterms give 0 for a unique combination."

"Karnaugh Map (K-Map) is a simple and visual method to simplify Boolean expressions."

K-Maps : up to 2, 3, 4 Variables

"By grouping adjacent 1's (in powers of 2), we can obtain the simplest Boolean expression."

1. Introduction

- K-Map (Karnaugh Map) is a graphical method used to minimize Boolean functions.
- It reduces the given Boolean expression to its simplest form (SOP or POS).
- It is easier and faster than algebraic simplification for small number of variables (up to 4 variables).
- Adjacent cells differ in only one variable.
- Group the 1's (for SOP) or 0's (for POS) in powers of 2 (1, 2, 4, 8, ...).
- Use don't care conditions (X) if given.

2. General Rules for K-Maps

- Group only adjacent 1's (for SOP) or adjacent 0's (for POS).
- Number of cells in a group must be a power of 2 (1, 2, 4, 8, ...).
- Groups can be horizontal, vertical or rectangular.
- Groups can wrap around the edges.
- Each 1 must be included in at least one group.
- Choose the largest possible groups.
- Overlapping groups are allowed.
- For a group of 2^n cells, n variables are eliminated from the term.

Group Sizes and Number of Variables

Number of Variables	K-Map Size	Possible Groups
2	2×2	1, 2, 4
3	2×4	1, 2, 4, 8
4	4×4	1, 2, 4, 8, 16

3. 2-Variable K-Map

	B	0	1	
A	0	m_0	m_1	
	1	m_2	m_3	

Cell expressions:
 $m_0 = A'B'$
 $m_1 = A'B$
 $m_2 = AB'$
 $m_3 = AB$

Example : Simplify $F(A, B) = \sum m(0, 1, 2)$

	B	0	1	
A	0	1	1	
	1	1	0	

Groups :
 $\bullet m_0, m_1 \rightarrow A'$
 $\bullet m_0, m_2 \rightarrow B'$
 $\therefore F = A' + B'$

4. 3-Variable K-Map

	BC	00	01	11	10	
A	0	m_0	m_1	m_3	m_2	
	1	m_4	m_5	m_7	m_6	

Example : Simplify $F(A, B, C) = \sum m(0, 1, 3, 5, 7)$

	BC	00	01	11	10	
A	0	1	1	1	0	
	1	0	1	1	0	

Groups :
 $\bullet m_0, m_1 \rightarrow \bar{A}\bar{B}'$
 $\bullet m_1, m_3 \rightarrow \bar{A}B'$
 $\bullet m_5, m_7 \rightarrow AC$
 $\therefore F = \bar{A}\bar{B}' + \bar{A}B' + AC$

5. 4-Variable K-Map

	CD	00	01	11	10	
AB	00	m_0	m_1	m_3	m_2	
	01	m_4	m_5	m_7	m_6	
	11	m_{12}	m_{13}	m_{15}	m_{14}	
	10	m_8	m_9	m_{11}	m_{10}	

Example : Simplify $F(A, B, C, D) = \sum m(0, 1, 2, 5, 6, 8, 9, 10)$

	CD	00	01	11	10	
AB	00	1	1	0	1	
	01	0	1	0	1	
	11	0	0	0	0	
	10	1	1	0	1	

Groups :
 $\bullet m_0, m_2, m_8, m_{10} \rightarrow B'D'$
 $\bullet m_0, m_1 \rightarrow \bar{A}\bar{B}'C'$
 $\bullet m_8, m_9 \rightarrow AB'C'$
 $\bullet m_{11}, m_{15} \rightarrow A'C'D$
 $\bullet m_2, m_6 \rightarrow A'CD$
 $\therefore F = B'D' + \bar{A}\bar{B}'C' + AB'C' + A'C'D + A'CD$

6. Important Cases

- (i) 4-cell group (eliminates 2 variables)
- | | |
|---|---|
| 1 | 1 |
| 1 | 1 |
- $\Rightarrow$ Result is one term (with 2 variables eliminated)
- (ii) 8-cell group (eliminates 3 variables)
- | | | | |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 |
- $\Rightarrow$ Result is one term (with 3 variables eliminated)
- (iii) 16-cell group (eliminates 4 variables)
- | | | | |
|---|---|---|---|
| 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 |
- $\Rightarrow$ Result is constant 1 (all variables eliminated)

7. Don't Care Conditions (X)

- Don't care conditions can be treated as either 0 or 1, whichever gives a larger group.
- They help in further simplification.

Example : $F(A, B, C) = \sum m(0, 1, 3, 7) + d(2, 5)$

	BC	00	01	11	10	
A	0	1	1	1	X	
	1	0	X	1	0	

$\therefore F = A' + BC$

8. K-Map for POS (Grouping 0's)

- For Product of Sums (POS), group the 0's instead of 1's.
- Follow the same rules as for grouping 1's

Example : $F(A, B) = \prod m(0, 2, 3)$

	B	0	1	
A	0	0	1	
	1	0	0	

$\therefore F = (A + B)(A' + B)$

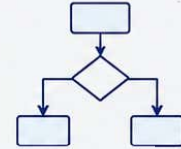
9. Key Takeaways

- K-maps provide a simple and efficient method to minimize Boolean expressions.
- Applicable for up to 4 variables.
- Group adjacent 1's (SOP) or 0's (POS) in powers of 2.
- Use don't care conditions to get larger groups.
- The simplified expression will have minimum number of terms and literals.

Conclusion :

K-Maps are a powerful and convenient method to simplify Boolean expressions up to 4 variables and are widely used in digital logic design.

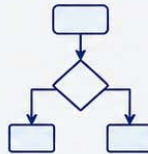
1010101010
0101010101
1010101010
0101010101



Unit - IV



10110
01001
10101



"A microcomputer is a complete computer system built around a microprocessor."

Microprocessor : Block Diagram of Microcomputer

"A microcomputer consists of CPU (microprocessor), memory, I/O devices and interconnection buses."

1. Introduction

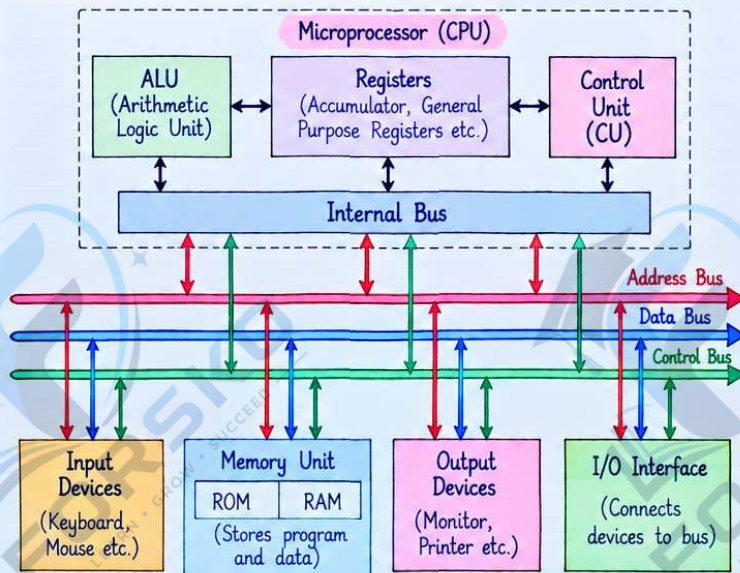
- A microcomputer is a small scale computer system designed around a microprocessor.
- It contains CPU, memory, input/output devices and system buses.
- It can perform data processing, storage and control functions.
- Examples : Personal computer (PC), Laptop, Embedded system etc.

2. Components of a Microcomputer

- Microprocessor (CPU)**
 - Performs arithmetic, logical and control operations.
- Memory Unit**
 - Stores programs and data (RAM, ROM).
- Input Devices**
 - Used to input data and instructions (Keyboard, Mouse).
- Output Devices**
 - Used to display the result (Monitor, Printer).
- System Buses**
 - Address bus, Data bus and Control bus for interconnection.

"All the units are interconnected through system buses."

3. Block Diagram of a Microcomputer



5. Functions of Buses

Bus	Function	Direction	Width
Address Bus	Carries address of memory or I/O device	Unidirectional (CPU → Memory/I/O)	n bits (e.g. 16/20/32)
Data Bus	Carries data between CPU, memory and I/O	Bidirectional	8/16/32 bits
Control Bus	Carries control signals (Read, Write, Interrupt etc.)	Bidirectional	Various lines

6. Key Points

- Microcomputer uses a single microprocessor chip.
- All units are connected through system buses.
- It is smaller, cheaper and easier to use than mainframe computers.
- Used in PCs, workstations and embedded systems.
- Data Bus** : Carries data (bidirectional).
- Control Bus** : Carries control signals (Read, Write, etc.).

Conclusion :

A microcomputer is a complete computer system with CPU, memory, I/O devices and buses working together to perform data processing tasks.

4. Explanation of Blocks

1. ALU (Arithmetic Logic Unit)

- Performs arithmetic operations (add, sub, etc.) and logical operations (AND, OR, etc.).

2. Registers

- Temporary storage inside CPU (e.g., Accumulator, Program Counter, Stack Pointer).

3. Control Unit

- Controls and coordinates the operations of all units.

4. Memory Unit

- ROM** : Stores permanent instructions.
- RAM** : Stores data and programs temporarily.

5. Input / Output Devices

- Input** : Keyboard, Mouse, Scanner etc.
- Output** : Monitor, Printer, Speaker etc.

6. I/O Interface

- Acts as an interface between I/O devices and the system bus.

7. System Buses

- Address Bus** : Carries address from CPU to memory/I/O (unidirectional).

"Evaluation of a microprocessor means analyzing its performance, features and suitability for a given application."

Evaluation of Microprocessor

"A good microprocessor is the one which gives higher performance, reliability and required features at a lower cost."

1. Introduction

- Evaluation of a microprocessor means comparing and analyzing different parameters to decide how good and efficient a processor is for a particular application.
- It helps in selecting the right processor based on performance, cost and features.
- The evaluation is done using factors such as clock speed, word length, memory capacity, instruction set, execution time, power consumption, etc.

2. Important Evaluation Factors

- Word length** - Number of bits processed at a time (8-bit, 16-bit, 32-bit, 64-bit).
- Clock speed** - Frequency of operation (measured in MHz or GHz).
- Instruction set** - Types and number of instructions available.
- Addressing capability** - Amount of memory that can be addressed.
- Execution time** - Time taken to execute an instruction.
- Throughput** - Number of instructions executed per second (MIPS).
- Power consumption** - Important for portable and embedded systems.
- Cost** - Price of the microprocessor.
- Peripheral support** - Availability of I/O interfaces (UART, USB, etc.).
- Reliability** - Stability and error handling.

3. Performance Parameters

(i) Clock Speed (f)

- Measured in Hertz (Hz).
- Indicates how many clock cycles per second the processor can execute.
- Higher clock speed generally means faster processing.

(ii) Instruction Execution Time (T)

$$T = \frac{\text{Number of clock cycles}}{\times \text{Time period of clock}}$$

- Time period of clock = $\frac{1}{f}$
- So, $T = \frac{(\text{Number of cycles})}{f}$

(iii) MIPS (Million Instructions Per Second)

$$\text{MIPS} = \frac{\text{Instruction count}}{\text{Execution time}} \times 10^6$$

(iv) Throughput

- Number of instructions executed per second.
- Measured in IPS (Instructions Per Second).
- Higher throughput means better performance.

7. Advantages of Proper Evaluation

- Helps in selecting the right processor for the application.
- Ensures better performance and efficiency.
- Reduces cost and power consumption.
- Improves system reliability and compatibility.

4. Example Calculation

A microprocessor executes an instruction in 4 clock cycles. Its clock speed is 5 MHz. Find the instruction execution time and MIPS.

Given :

Clock cycles per instruction = 4

Clock speed (f) = 5 MHz = 5×10^6 Hz

$$\text{Time period of clock} = \frac{1}{f} = \frac{1}{5 \times 10^6} \\ = 0.2 \times 10^{-6} \text{ s} = 0.2 \mu\text{s}$$

$$\text{Execution time } (\bar{T}) = 4 \times 0.2 \mu\text{s} \\ = 0.8 \mu\text{s}$$

If the instruction count is 1 million :

$$\text{MIPS} = \frac{1 \times 10^6}{0.8 \times 10^{-6}} \times 10^6 \\ = 1.25 \text{ MIPS}$$

5. Comparison of Microprocessors

Parameter	8085	8086	80286	80386
Word length	8-bit	16-bit	16-bit	32-bit
Clock speed	3 MHz	5-10 MHz	6-12 MHz	16-33 MHz
Max memory addressable	64 KB	1 MB	16 MB	4 GB
Instruction set	74	117	134	183
MIPS (approx.)	0.5	1-2	2-5	10-20
Operating mode	Single	Single	Real & Protected	Real & Protected
Applications	Simple systems	PC/XT	AT systems	High performance

6. Evaluation Criteria (Selection Guidelines)

- Choose a processor with sufficient word length for the application.
- Higher clock speed and MIPS for better performance.
- Adequate memory addressing capability.
- Rich instruction set and addressing modes.
- Low power consumption (for portable/embedded applications).
- Availability of required I/O interfaces.
- Cost should be within budget.
- Consider reliability and long-term support.

8. Conclusion

Evaluation of a microprocessor is essential to compare its features and performance. By considering parameters like clock speed, word length, MIPS, memory capacity, power consumption and cost, we can select the most suitable processor for a given application.

"The best microprocessor is not always the fastest one, but the one that best meets the requirements of the application."

"Architecture tells us how the parts are organized, connected and work together inside the 8085."

Architecture Of 8085 Microprocessor

"8085 is an 8-bit microprocessor developed by Intel in 1976, widely used in educational and embedded systems."

1. Introduction

- The 8085 is an 8-bit microprocessor developed by Intel in 1976.
- It belongs to the MCS-80 family.
- It can process 8-bit data at a time and can address 64 KB (2^{16}) memory locations.
- It is a general purpose microprocessor used in various applications.

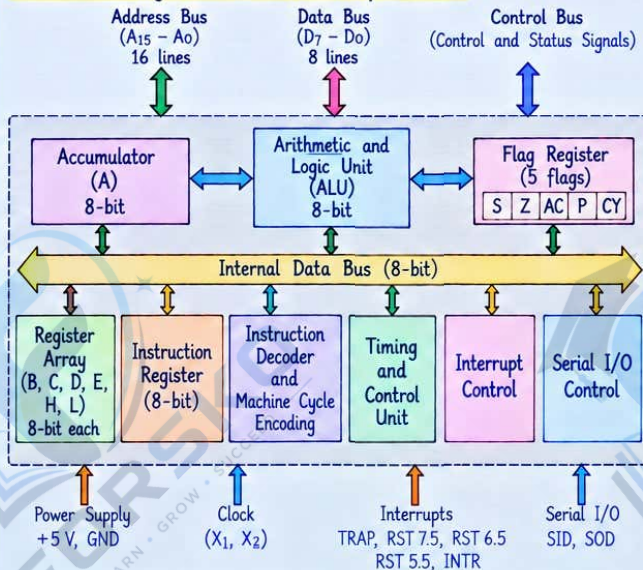
2. Main Features

- 8-bit data bus
- 16-bit address bus (can address 64 KB memory)
- 40 pins
- Operates on +5 V DC power supply
- Clock frequency : 3 MHz (typical)
- Supports 74 instructions
- Has 5 interrupt lines
- Serial I/O communication
- Suitable for small and medium scale applications

3. Architecture Overview

- The architecture shows the internal organization of the 8085.
- It consists of ALU, registers, control unit, instruction register and decoder, timing and control unit, buses and I/O control etc.
- All these units work together to fetch, decode and execute instructions.

4. Block Diagram of 8085 Microprocessor



5. Function of Each Block

Block / Unit	Function
ALU	Performs arithmetic and logic operations (add, subtract, AND, OR, etc.).
Accumulator (A)	An 8-bit register used to store the result of ALU operations.
Flag Register	Stores the status of the result (S, Z, AC, P, CY).
Register Array	General purpose registers B, C, D, E, H, L (8-bit each).
Instruction Register	Holds the opcode of the current instruction.
Instruction Decoder	Decodes the instruction and generates control signals.
Timing and Control Unit	Generates timing and control signals for all operations.
Interrupt Control	Handles external and internal interrupts.
Serial I/O Control	Manages serial input (SID) and serial output (SOD).
Buses	Address bus (16-bit), Data bus (8-bit) and Control bus.

6. Registers in 8085

Register	Function
Accumulator (A)	8-bit register, stores result of ALU operations.
B, C, D, E, H, L	General purpose 8-bit registers.
Program Counter (PC)	16-bit register, holds address of next instruction.
Stack Pointer (SP)	16-bit register, points to top of stack.
Instruction Register (IR)	8-bit register, holds current instruction.
Flag Register	5 flags : Sign (S), Zero (Z), Auxiliary Carry (AC), Parity (P), Carry (CY).

7. Buses in 8085

Bus	Function
Address Bus (16-bit)	Carries the address of memory or I/O location. (A ₁₅ - A ₀)
Data Bus (8-bit)	Carries data between CPU, memory and I/O devices. (D ₇ - D ₀)
Control Bus	Carries control and status signals like RD, WR, ALE, I/O/M, S ₀ , S ₁ , etc.

8. Interrupts in 8085

Interrupt	Type	Description
TRAP	Non-maskable	Highest priority, edge and level triggered.
RST 7.5	Maskable	Edge triggered.
RST 6.5	Maskable	Level triggered.
RST 5.5	Maskable	Level triggered.
INTR	Maskable	General purpose interrupt.

9. Important Points

- 8085 is an 8-bit microprocessor with 40 pins.
- It can address 64 KB memory.
- It has a simple and flexible architecture.
- Used widely in education and embedded systems.
- It supports both memory-mapped and I/O-mapped I/O.
- It accepts interrupts and provides serial I/O.

10. Conclusion

"The 8085 microprocessor has a simple and efficient architecture, making it a popular choice for learning and basic embedded applications."

"8085 is a simple, beautiful 8-bit microprocessor, which laid the foundation for modern processors."

8085 IN MICROPROCESSOR

Architecture - Features - Pin Diagram - Instructions - Applications

"Small in size, but a big step in the history of computing."

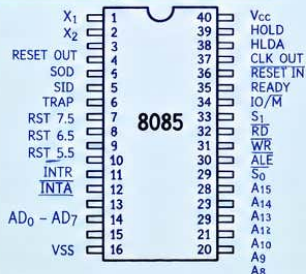
1. Introduction

- 8085 is an 8-bit microprocessor developed by Intel in 1976.
- It belongs to the MCS-80 family (8080, 8085, 8086, etc.).
- It can process 8-bit data at a time and can address 64 KB (2^{16}) memory locations.
- It is a general purpose microprocessor used in many applications.
- 8085 is an improved version of 8080 with more on-chip features like serial I/O, additional instructions and better interrupt structure.

2. Main Features

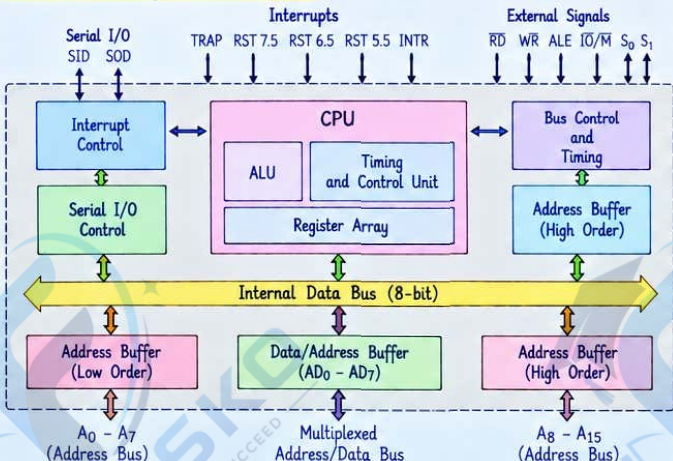
- 8-bit data bus and 16-bit address bus.
- Can address 64 KB memory (0000H-FFFFH).
- Operates on +5V single power supply.
- Clock frequency : 3 MHz (typical).
- Supports 74 instructions.
- Has 5 hardware interrupts.
- In-built serial I/O control.
- On-chip clock generator.
- Multiplexed address and data bus ($AD_0 - AD_7$).
- Available in 40-pin DIP package.
- Low power and high reliability.

3. Pin Diagram of 8085



8085 is available in 40-pin DIP package.

4. Block Diagram of 8085



7. Addressing Modes in 8085

Addressing Mode	Example
Immediate	MVI A, 3AH
Register	MOV B, C
Direct	LDA 2050H
Register Indirect	MOV A, M
Implied	CMA

10. Applications of 8085

- Used in microcomputer systems, embedded systems and industrial control.
- Applied in traffic lights, home appliances, instrumentation, data acquisition systems.
- Widely used in education and research for learning microprocessor concepts.
- Still important for understanding the basics of modern processors.

"8085 is not just a microprocessor, it is the beginning of a revolution in computing."

5. Registers in 8085

Register	Size	Function
Accumulator (A)	8-bit	Used to store data and result of ALU operations.
B, C, D, E, H, L	8-bit each	General purpose registers. Can be used individually or as register pairs (BC, DE, HL).
Program Counter (PC)	16-bit	Holds address of next instruction to be executed.
Stack Pointer (SP)	16-bit	Points to top of the stack.
Flag Register	8-bit	Stores status flags (S, Z, AC, P, CY).
Instruction Register	8-bit	Holds the current instruction.
Temporary Register	8-bit	Used internally by CPU.

6. Flag Register

Bit	D7	D6	D5	D4	D3	D2	D1	D0
Flag	S	Z	-	AC	-	P	-	CY
S	: Sign flag			(1 if result is negative)				
Z	: Zero flag			(1 if result is zero)				
AC	: Auxiliary Carry			(1 if carry from bit 3 to bit 4)				
P	: Parity flag			(1 if result has even parity)				
CY	: Carry flag			(1 if carry out from MSB)				
-	: Unused							

8. Instruction Set

- 8085 has 74 instructions.
- Instructions are classified as:
 - Data Transfer Instructions
 - Arithmetic Instructions
 - Logical Instructions
 - Branch (Jump) Instructions
 - Stack, I/O and Machine Control Instructions

9. Timing and Machine Cycle

- The 8085 uses machine cycles to execute instructions.
- Each machine cycle consists of one or more T-states (clock cycles).
- Basic machine cycles:
 - Opcode Fetch
 - Memory Read
 - Memory Write
 - I/O Read
 - I/O Write

11. Conclusion

The 8085 microprocessor is a simple, reliable and efficient 8-bit processor. Its flexible architecture, rich instruction set and easy interfacing make it a popular choice for learning and embedded applications. It forms the foundation for modern microprocessors.